



Sanskrit LLM: Proving the Need for a Panini Sutra Rule-Based AI Model by Benchmarking the Capabilities of Existing LLMs

Shreya Srikant^{1(✉)}, Vismaya Murali¹, Varsha Badri¹, S. S. Shylaja¹, and Mukund Rao²

¹ Department of CSE, PES University, Bengaluru, India
srikantshreya@gmail.com, vismayamurali05@gmail.com,
varshabadri19@gmail.com, shylaja.sharath@pes.edu

² Amazon Web Services, Austin, TX, USA
mukund1989@gmail.com

Abstract. In recent times, there have been significant advancements in the field of LLM research, especially in the Indic LLM sphere. Sanskrit, being an integral part of India's culture and heritage, forms the root of many other Indian languages. Current large language models often lack a deep understanding of language structure and meaning, especially for Sanskrit, leading to outputs that can be grammatically incorrect and logically flawed or biased. In this paper, we propose the Panini Sutra-based AI model that draws inspiration from Panini's Ashtadhyayi, an ancient treatise on Sanskrit grammar consisting of nearly 4,000 sutras. To signify the need for this model, we benchmark the Indic Language Capabilities of existing Large Language Models by varying certain parameters. These include benchmarking outputs from models using different tokenizers on the same dataset and fine-tuning an existing open-source model to highlight the lack of an annotated dataset and poor linguistic capabilities. We used character-level and sub-word tokenizers like Google's SentencePiece and OpenAI's Tiktokenizer. The Gemma 2B model is leveraged for fine-tuning and is evaluated on the accuracy of translation between Sanskrit and English. We reckon the results obtained can broaden the dimensions and support the development of Sanskrit-based LLM.

Keywords: LLM · Indic LLM · Sanskrit NLP · Panini Sutra · Tokenization · Fine-tuning · Gemma 2B · Translation · Benchmarking

1 Introduction

Sanskrit, an ancient Indo-European language, is known for its rich literary and philosophical works. It's the language of the Vedas, Upanishads, and other sacred texts. It spans diverse fields- science, literature, mathematics, astronomy, philosophy, and more, highlighting its rich intellectual depth and its foundational influence on other languages. Sanskrit's grammar is complex and was expertly organized by the ancient scholar Panini in his work, the Ashtadhyayi. Written around the 4th century BCE, Ashtadhyayi comprises nearly 4,000 concise rules called sutras. These are arranged into eight chapters, each divided into four sections. Panini's work carefully explains Sanskrit's sounds, word forms, and sentence structures, providing a clear and detailed guide

to the language. His meticulous work not only preserves Sanskrit but also is a significant achievement in studying languages.

There has been a growing scope for research in Indic language models in recent years. With the increasing use of ChatGPT, understanding and comprehending English has become easier for everyday English speakers. In culturally and linguistically diverse countries like India, people want to learn information on the internet in their regional languages without language acting as a barrier.

Models like OpenHathi and others are emerging with multilingual capabilities in 22 main languages, accounting for only 1.27% of the total languages. Yet there remain very few dedicated efforts for Sanskrit, one of the oldest known languages, renowned for its unparalleled richness and historical significance. With the advent of Generative AI, there has been a significant push towards creating large language models that can comprehend and generate highly accurate human-like language. However, these advancements have largely focused on widely spoken languages, leaving a gap in developing models for less commonly used but culturally and historically rich languages like Sanskrit. The intricate structure and depth of Sanskrit present unique challenges that are not adequately addressed by current LLMs. This paper aims to bridge this gap by proposing a Panini Sutra-based large language model inspired by the Ashtadhyayi, a comprehensive treatise on Sanskrit grammar.

Existing LLMs, including those with multilingual capabilities, often fall short in understanding and generating Sanskrit due to a lack of annotated datasets and the complexity of the language's grammatical rules. By leveraging Panini's meticulous grammatical framework, we aim to enhance the linguistic capabilities of LLMs specifically for Sanskrit. Our approach involves benchmarking the performance of various tokenizers on Sanskrit datasets and fine-tuning the Gemma 2B model to improve translation accuracy between Sanskrit and English. This research not only highlights the need for more dedicated efforts in the development of Sanskrit-based LLMs but also sets a precedent for incorporating ancient grammatical knowledge into modern AI models.

2 Challenges

2.1 Complex Grammar and Morphological Structures

A significant barrier to the development of Sanskrit LLM models is the intrinsic complexities of the grammar characterized by an elaborate morphological structure that includes eight cases, three numbers (singular, dual, plural), and three genders (masculine, feminine, neuter), leading to a vast array of noun, pronoun, and adjective forms. Additionally, the language's verbal conjugation is intricate, consisting of verb roots with numerous derivatives.

Like many South Indian languages, Sanskrit uses sandhi extensively. Sandhi is of two kinds external sandhi (rules which govern the sandhi between two words) and internal sandhi (rules which govern the sandhi within a word involving two or more morphological segments). Sandhi, which means "joining" or "junction", refers to the changes that happen at the junction of different sounds in a language. While every language has its own sandhi rules, Sanskrit sandhi is particularly complex and distinctive.

Example: “rāmālaya”

The word “rāmālaya” illustrates this complexity through its possible segmentations:

Internal Sandhi Variations:

ra + amālaya
 ra + āmālaya
 rā + amālaya
 rā + āmālaya
 rāma + alaya
 rāma + ālaya
 rāmā + alaya
 rāmā + ālaya

Each of these segmentations reflects the interplay of sandhi rules and the rich morphological inflection of Sanskrit, demonstrating the challenges in developing language models for such a grammatically complex language.

2.2 Archaic Features and Historical Usage

Sanskrit initially originated as Vedic Sanskrit between 1700 and 1200 BCE and was traditionally preserved orally through Vedic chanting. This ancient form of Sanskrit is the language of the Vedas, the earliest sacred texts of Hinduism which can be found in the Rigveda, a compilation of 1,028 hymns. Around 500 BCE, the scholar Panini standardized Vedic Sanskrit into what is now known as Classical Sanskrit by establishing its grammatical rules.

The language of the Rigveda includes many archaic features not found in later Sanskrit texts, such as extensive use of the subjunctive mood, certain archaic phonetic forms, and a freer word order. These features indicate its early stage of development and present additional challenges for modern interpretation and translation. The model must learn to differentiate and correctly interpret these older forms alongside the more standardized Sanskrit, requiring a well annotated training corpus.

2.3 Polysemy and Contextual Nuance

Another layer of complexity arises from polysemy, where a single word can have multiple meanings depending on the context, requiring a deep understanding of nuance and syntax.

Example: When translating the Sanskrit sentence “यत्र धर्मस्तत्र जयः” (“Where there is Dharma (Righteousness), there is victory”) using the original Gemma 2b instruct model, the output was: “Wherever there is religion, there is joy.” In this case, the word “धर्म”(dharma) was incorrectly interpreted as “religion”. The word “धर्म” in Sanskrit has various meanings like righteousness, duty, law, religion, and more. Thus, the correct meaning of the word depends on the context of the sentence in which it is used. Existing LLMs often lack the necessary language knowledge and sufficient training examples that cover the diverse senses of each polysemous word, resulting in challenges in achieving accurate disambiguation (selecting correct sense of a word).

2.4 Lack of Annotated Data

Languages like English have vast amounts of data readily available, but Indic languages, particularly Sanskrit, have significantly less. State-of-the-art models trained on billions of tokens for various tasks mainly focus on English, which is not feasible for Sanskrit due to limited resources. Developing LLMs for Indian languages faces challenges like limited digital content, diverse scripts, and complex grammar, compounded by resource constraints and a lack of standardized tools and collaboration. Overcoming these obstacles requires collaborative data collection, resource allocation, and community involvement.

The lack of extensively annotated datasets can lead to insufficient exposure to diverse linguistic structures, resulting in lower model performance. Existing models struggle with extensive morphological variations, causing errors in understanding and generating correct forms. Complex sandhi rules can complicate tokenization and parsing, affecting comprehension and generation.

To gain more insights into the shortcomings of the current models and evaluate the effectiveness of these models, we have adopted a benchmarking process for evaluating multiple Sanskrit language models to underscore the need for a Panini's rule-based engine. By highlighting these limitations, we aim to demonstrate how a rule-based approach can enhance the performance and accuracy of Indic LLMs.

3 Methodology

The approaches we followed to benchmark the results of the experiments are:

1. **Zero-shot prompting** on the Rudra 7B model, a fine-tuned Gemma 7B model that takes a word as input and gives a few lines about the word as output.
2. Creating a **pre-trained model from scratch** for 3 different tokenizers: Character level, SentencePiece, and Tiktokenizer, and benchmarking the outputs against BLEU and perplexity scores.
3. **Fine-tuning the Gemma 2B** instruction model using Low-Rank Adaptation (LoRA) Parameter-Efficient Fine-Tuning (PEFT) on translation tasks between Sanskrit and English to evaluate the accuracy of translated texts.

3.1 Pre Training from Scratch

Pre-training involves training a neural network model on a large corpus of data in an unsupervised fashion. This helps the neural network learn general patterns in the corpus. The parameters used are attached in Table 2. The steps followed are:

Dataset Description. The dataset used is referenced in Table 1.

Defining Hyperparameters. The hyperparameters defined are crucial for determining the model's performance, speed, and capacity. The Parameters we define are attached in Table 2.

Table 1. Pre-training process

Data corpus	Mahabharata dataset: 2 MB
Language	Sanskrit
Tokenizers	Character Level SentencePiece Tiktokenizer
Training and validation split	Train split: 80% Validation split: 20%
Optimiser	AdamW
Evaluation metrics	BLEU score Perplexity score

Table 2. Hyperparameters defined for pre-training

Parameters	Description	Value
Learning rate	Set for the optimizer while training	1e-3
max_iters	Maximum number of training iterations/steps	5000
n_embd	Number of embedding dimensions for the mapping space of input sequences	64 dimensions
n_head	Number of attention heads in the multi-head attention layer of the model. The attention mechanism allows the model to focus on certain parts of the input sequence	4
n_layer	This parameter sets the number of layers in the model (the number of decoder layers in our model)	4

Tokenization.

1. **Using character level tokenisation:** We create the encoder and decoder function from the list of characters from the Dataset. The list of characters has all the Devanagari alphabets along with special characters containing “ँ ं ः अ आ ई उ ऊ ऋ ए ऐ ओ औ क ख ग घ ङ च छ ज झ ञ ट ठ ड ढ ण त थ द ध न प फ ब भ म य र ल व श ष स ह ऽ ा ि िी उू ँ ऌ ऍ ऎ”. The Encoder takes the character and returns the integer corresponding to the character’s position in the list, and a tensor is created from the tokenized dataset. The Decoder gives the corresponding character for the integer index in the tensor.
2. **Using Google’s SentencePiece tokenizer:** We trained SentencePiece on the Mahabharata dataset, keeping the vocabulary size limit of 8000. It is a language-independent, unsupervised sub-word tokenizer for neural network text processing systems. Because it is language-independent, we can use this tokenizer for benchmarking.
3. **OpenAI’s Tiktokenizer:** Tiktokenizer is a fast byte pair encoding tokenizer. It is a subword tokenizer, that converts text into tokens based on the kind of encoding base model inputted, which in our case, was GPT-2.

Our approach involved pre-training a model using the three mentioned tokenizers. We then evaluated their performance on the same corpus of data to identify which tokenizer yields the best results. The choice of tokenizer plays a key role when dealing with Indian languages. Using the tokenizers, we create the encoding and decoding mappings between either the characters or subwords to integer tensors.

Self-attention Mechanism of the Transformer. We are using a decoder-only architecture, with the following transformer blocks - self-attention multi-head layer, feed-forward linear layer, layer normalization.

1. **Multi-Head Self-Attention layer:** We create a single Head of Self-Attention and process them in parallel to get the Multi-Head Self-Attention layer. For a single-head function, we use 3 linear layers (key, query, value) that will project the input tensor into a lower dimension to compute attention scores by taking the dot product of key and query projections and normalizing it by the root of the embedding size. A triangular mask is applied to ensure predictions rely only on previous and current inputs, avoiding future tokens. Softmax and dropout are then used to normalize scores, and the weighted aggregation of values generates the output tensor. Concatenating multiple single-head layers and then projecting onto the original embedding size gives the output of a multi-head layer.
2. **Feed Forward layer:** A neural network with ReLU - Rectified Linear Units activation and dropout regularisation where ReLU introduces non-linearity between linear layers so the model can learn complex patterns of data.

Model Training and Generating Text Using The Transformer Language Model.

The embedding layer is defined for the tokens and for the position of the tokens. We include all the transformer blocks here in a sequence and then find logits (Raw probabilities generated by the final layer of the neural network before normalisation is applied) of the next token using an embedding lookup table. Then the softmax function is used to normalize the raw logits into probabilities. Provided an input sequence, and the number of tokens to be generated, it uses the lookup table and probability distribution of the model, to get the next set of predicted tokens. Then we optimize the model using the AdamW optimizer and we evaluate the model by calculating loss, and gradients through backpropagation and updating model parameters. AdamW optimization is a stochastic gradient descent method that is based on adaptive estimation of first-order and second-order moments with an added method to decay weights.

Finally, for each training iteration, we calculate the loss of each split dataset by calculating logits and the loss of each batch of inputs by evaluating `eval_iters` times. After this, text can be generated (refer to Appendix B for Model Outputs).

3.2 Fine-Tuning Approach

The Gemma-2b instruct model was considered as the base model for fine tuning, to translate Sanskrit texts into English. Fine-tuning involves training a pre-trained language model on a specific dataset to enhance its performance for a particular task. The

model was fine-tuned using a custom dataset consisting of 20000 aligned Sanskrit-English sentence pairs (refer to Table 3).

Table 3. Fine-tuning approach

Data corpus	Sanskrit-llama (source: Huggingface) 4.6 MB
Language	Sanskrit, English
Tokenizers	Gemma 2b instruct model tokenizer
Training and validation split	Train split: 80% Validation split: 20%

Dataset Description. The dataset used is defined in Table 3.

Training Parameters. The training parameters are defined in Table 4.

Table 4. Training parameters

Parameter	Description	Value
Gradient Accumulation Steps	Accumulation of gradients over multiple steps to reduce memory consumption	4
Warmup Steps	Initial training steps with a gradual increase in learning rate for stabilization	0.03
Learning Rate	Step size during gradient descent to control parameter updates	2e-4

Fine-Tuning Process.

1. **Low-Rank Adaptation (LoRA):** This technique reduces the number of trainable parameters in large language models while maintaining or even improving performance. This method was used in fine-tuning the Gemma 2b instruct model.
2. The Gemma-2b instruct model was trained on the T4 GPU provided by **Colab Notebook for around three hours.**
3. We perform supervised finetuning using the SFTTrainer.

The SFTTrainer from the trl (Transformer Reinforcement Learning) library is specialized for training transformer models with a focus on fine-tuning pre-trained language models.

The fine-tuning process was conducted in three discrete rounds, each comprising 500 training steps.

1. **1st Round:** Initial training focused on initializing model weights and adapting to the Sanskrit-English dataset.
Loss: 2.6046145145893096
2. **2nd Round:** Continued fine-tuning with extended training duration to further refine translation performance.
Loss: 2.1217723684310914

3. **3rd Round:** The final round aimed at optimizing translation accuracy and minimizing loss until convergence.
Loss: 1.7513068828582763 (at 1500 steps)
4. After an additional 100 steps, the loss was reduced to 0.674277121424675

Upon completing 1600 training steps, the Gemma-2b instruct model achieved a final training loss of 0.674277121424675. This metric indicates the model’s proficiency in minimizing errors and discrepancies between predicted and actual translations.

Performance Evaluation. The model’s performance was evaluated on various Sanskrit texts from the dataset. The translations exhibited minimal context understanding and showed inaccuracies or lacked fluency, highlighting areas for potential improvement (refer to Appendix C for Model Outputs).

3.3 Rudra-7B Model Experiment

The Rudra-7B model is an existing open-source model used for text generation in the Sanskrit language (refer to Table 5). It was trained for 29 h in total. Zero-shot prompting on the model was conducted with different inputs to analyze all kinds of outputs (refer to Appendix A for Model Outputs).

4 Evaluation Metrics

4.1 BLEU Score

The BLEU (Bilingual Evaluation Understudy) score is a metric used to evaluate the quality of machine-generated translations by comparing them to human-generated reference translations. It measures how many n-grams (contiguous sequences of words) in the candidate translation match those in the reference translation. The BLEU score ranges from 0 to 1, with higher scores indicating closer matches to the reference.

4.2 Perplexity Score

Perplexity is a measure of how well a language model predicts a sample, calculated as the exponential of the loss. It tells us how “confused” the model is when guessing the next word in a sequence. A lower perplexity means the model is better at making accurate predictions, indicating the text is easier for the model to understand. Essentially, it’s a way to gauge the performance of the model in understanding and generating text. A relatively low perplexity score(< 50) indicates a good score.

Table 5. Rudra 7B model

Data Corpus	Ancient Sanskrit Indian texts and Wikipedia articles
Language	Sanskrit
Base Model	Gemma-7B
Fine-tuning technique	QLoRA (Quantized Low-Rank Adaptation)

5 Inferences

5.1 Pre-trained Model

Different tokenizers used and benchmarked output quality from model can be found in Table 6.

Table 6. Benchmarked outputs of different tokenisers from pre-trained model

	Type	Perplexity score	BLEU score
Temperature: 1 Top_k value: 30	Character encoding	6.1998	0.2096
	SentencePiece	57.8897	0.2354
	Tiktoken	3.6344	0.1588
Temperature: 0.8 Top_k value: 20	Character encoding	5.9516	0.2485
	SentencePiece	47.7948	0.2352
	Tiktoken	3.4948	0.2098
Temperature: 0.7 Top_k value: 10	Character encoding	6.2554	0.2218
	SentencePiece	73.9025	0.1515
	Tiktoken	3.9484	0.1658

5.2 Defining Temperature and Top_k Value

1. **Temperature:** Temperature changes the randomness of predictions by flattening or increasing the probability density of the model. Logits are scaled before softmax normalization. A low temperature, typically less than 1.0 means the randomness is less, and the highest probable tokens will be generated with less diversity amongst them.
2. **Top-k value:** Top-k sampling is used to limit the number of tokens to sample to the top-k highest probability. All other tokens are discarded. A low top-k value (<10) can make the outputs less diverse as tokens are restricted so outputs can become repetitive but it also makes it more confident. A higher value of around 50 makes outputs more diverse and less confident if low-probability tokens are considered too.

In the above method, we see three different values for top-k and temperature being used. According to the different parameters used, a highly confident output would be obtained with values temperature=0.7 and top-k=10 but for a balance between confidence and diversity, a combination of temperature=1 and top-k=30 is recommended.

6 Observations

6.1 Character Encoding

1. **Perplexity:** Shows relatively low values across different settings, indicating strong predictive performance.
2. **BLEU Score:** The highest score is 0.2485 at a temperature of 0.8 and top.k value of 20, suggesting this configuration is the most effective for generating high-quality translations.

6.2 SentencePiece

- 1. **Perplexity:** Higher across different settings, indicating weaker predictive performance compared to the other tokenizers.
- 2. **BLEU Score:** The best score is 0.2353 at temperature 1 and a top.k value of 30. Despite the higher perplexity, the BLEU score is reasonably high, indicating that while predictions might not be the most accurate, the quality of the generated text is still relatively good.

6.3 Tiktoken

- 1. **Perplexity:** Consistently the lowest, which indicates it excels at predicting the next token in a sequence.
- 2. **BLEU Score:** Peaks at 0.2098 with a temperature of 0.8 and top.k value of 20. This shows a balance between prediction accuracy and translation quality, but it is not as high as Character Encoding under optimal settings.

To summarize the observations made and visualize graphs, Table 7, Fig. 1, Fig. 2. Different metrics highlight different strengths of your tokenizers. Tiktoken is highly reliable in predicting the next tokens, while Character Encoding balances prediction and generation quality well in specific settings. SentencePiece, though less reliable in

Table 7. Observations

Encoding Method	Prediction Quality (Perplexity)	Text Generation Quality (BLEU)
Tiktoken	Low	Lower
Character Encoding	Balanced	Best at certain settings
SentencePiece	High	Moderate

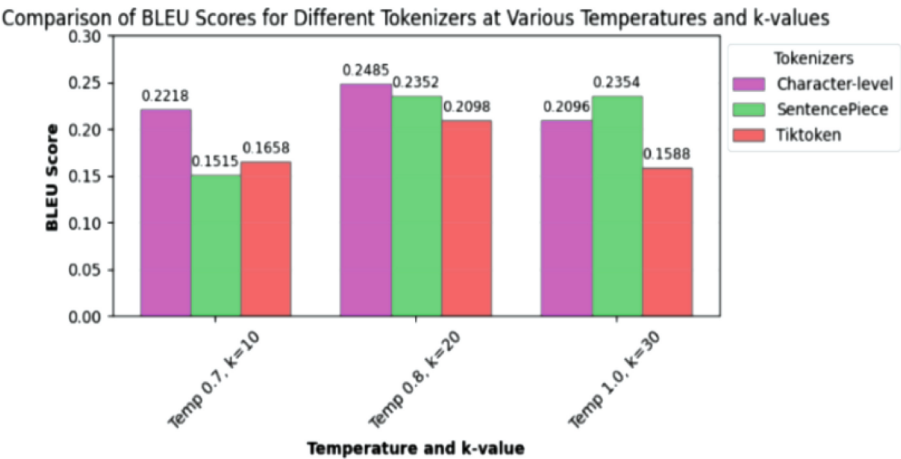


Fig. 1. BLEU score comparison

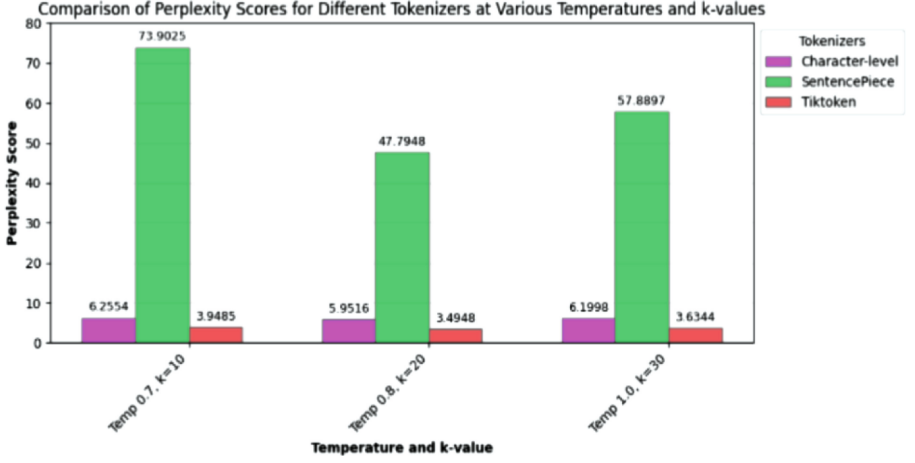


Fig. 2. Perplexity score comparison

prediction, still generates decent-quality text. From the above, we can conclude that a tokenizer that has the same reliability as tiktoken and also the attribute of having good contextual relation between tokens of SentencePiece would be a good tokenizer to use. Refer to Fig. 15 and Fig. 16 of Appendix D for further comparison of BLEU and Perplexity scores for different tokenizers across different temperatures.

Based on human evaluation, the outputs were found to be inaccurate and irrelevant. The text had several grammatical inconsistencies, syntax errors and incorrect sentence formations which made it hard to follow the meaning. Fine-tuned translations were misleading and failed to capture the context of the recognized words in the sentence. These results prove the need for a Panini rule-based Sanskrit Large Language Model.

7 Proposed Solution

Panini's Ashtadhyayi, written sometime around the 4th century BCE, is a full grammatical system for Sanskrit made up of nearly 4,000 brief and interdependent rules called "sutras". The sutras specify the organization of the language at the level of morphological, phonological, and syntactic structure with a precision approaching modern computational grammar formalisms. The system relies on a rule-generated generative process similar to that of contemporary production grammars, where words are built step by step through a series of transformations. His grammar has long been regarded not only as a linguistic wonder but as the ancestor of a rule-governed automaton, an embryonic basis of formal systems and even providing the stimulus for early computational linguistics.

The rule-based engine conceived from Panini's rule system is more than a post-processing correction layer; it is an inherently different entity that injects linguistic determinism into otherwise probabilistic language models. Unlike conventional boosts based on data scaling or hyperparameter tuning, this engine is theory-based and an intermediary validator and enhancer that acts both before and after LLM inference. In

preprocessing, it annotates and restructures raw Sanskrit inputs to grammatically standard forms. In postprocessing, it corrects outputs produced by the LLM by reapplying rule-based constraints from Panini's grammar, ensuring phonetic, syntactic, and semantic correctness. This two-way interface facilitates a deeper structural understanding of Sanskrit—solving sandhi, inflection, and polysemy—where statistical models are likely to reach their limits. The rule engine is thus not an incremental tweak but a fundamental architecture shift that injects explicit grammatical reasoning into LLM pipelines, a giant leap in the construction of linguistically informed AI systems for classical languages. See Fig. 3 for the design of the proposed Panini-based rule engine.

7.1 Architecture and Functionality

Data Processing and Annotation.

1. Sanskrit unannotated data can be processed through Panini's engine to produce clean, annotated data for which NLP toolkits would be leveraged. This annotated data can then be used to train LLMs, enhancing their linguistic accuracy and providing a method to generate more annotated datasets.
2. The annotated data, enriched with part-of-speech tagging, named entity recognition (NER), and identification of gender, case, and number, ensures comprehensive linguistic analysis. This improves the accuracy of language models by providing them with a deeper understanding of Sanskrit syntax and semantics.

User Interaction and Error Rectification.

1. The rule engine acts as a mediator between the user and the open-source LLM model. When a user inputs Sanskrit text, the rule engine analyzes it to ensure it follows grammatical rules, converting it to the appropriate format.
2. Similarly, when the LLM model generates a response, the output is passed through the rule engine to rectify errors, making the text contextually accurate and consistent.

Natural Language Processing Toolkits. Natural Language processing toolkits could be leveraged by having an additional layer in the rule engine for text preprocessing and syntactic, semantic, and morphological analysis in the following ways:

1. **Tokenization:** Segmenting Sanskrit texts into words, phrases, or other meaningful units.
2. **Part-of-Speech Tagging:** Identifying and tagging parts of speech (nouns, verbs, adjectives, etc.) in Sanskrit sentences.
3. **Sandhi Processing:** Applying rules for phonetic modifications (Sandhi) to ensure phonological correctness.
4. **Named Entity Recognition:** Identifying and classifying proper nouns, such as names of people, places, and organizations.

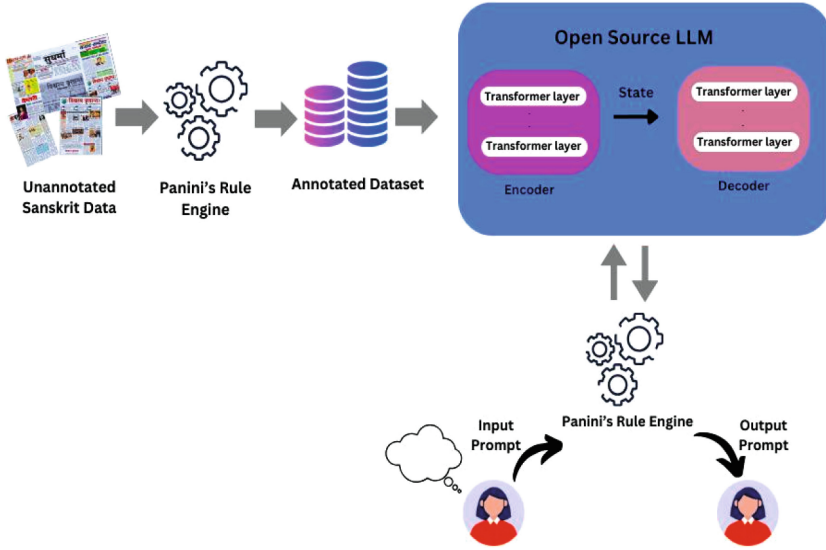


Fig. 3. Panini Sutra Based AI model architecture

7.2 How Processed Data Can Be Fed Into the LLM Architecture

Pre-training Phase. The annotated data produced by Panini's engine can be used as a high-quality corpus to pre-train the LLM ensuring that the model learns the fundamental linguistic rules and grammar of Sanskrit, enhancing its understanding and generating capabilities.

Fine-Tuning Phase. During fine-tuning, the annotated datasets with detailed tags (part-of-speech, Named Entity Recognition, gender, case, and number) can be used to further train the LLM on specific tasks. This helps the model adapt to more specific data, specialized linguistic nuances, and applications.

Rule-Based Post-processing. After the LLM generates text, the output can be passed through Panini's rule engine to correct any grammatical or phonological errors. This post-processing step ensures that the generated text adheres to traditional Sanskrit grammar rules, improving its accuracy and acceptability.

7.3 Applications and Benefits

1. **Decrypting Ancient Texts:** The Panini Sutra-based AI model can decrypt ancient texts and reconstruct missing manuscripts by integrating grammatical and phonological rules into an advanced AI framework.
2. **Enhanced Linguistic Understanding:** By incorporating these ancient rules, the model supports a deeper understanding of complex linguistic structures and facilitates multiple interpretations, considering the polysemy of the language.

- A brief, terse statement conveying essential knowledge.
- A collection of rules (e.g., Panini's *Ashtadhyayi* in grammar, Patañjali's *Yoga Sutras* in philosophy).

When prompted for *Sutra*, the output was as follows: “A thread is adorned by a thread; a thread shines by a thread.

That which is golden, gem-studded, and has silk in its midst;

A garland reaching the ear, that thread is like the crescent moon.

A thread is a collection of verses, greatly increasing auspiciousness.

A thread shines by a thread; that thread is like the crescent moon.

A thread shines by a thread; that thread is a collection of verses.

A thread shines by a thread; that thread is a collection of verses.

A thread shines by a thread; that thread is a collection of verses.

A thread shines by a thread; that thread is a collection of verses.

A thread shines by a thread; that thread is a collection of verses.

Thread, O thread!”

B Outputs of Pre-trained Model

यत्र स किं विद वयाशा पुत्र कथं परिचितः
 साहम एते विधिको ऽसमाद्य अतीक्ष्यः सुत्रीतस तदा
 विधिशा चेव संजायत परतिष्ठितः
 वयासंस्यामि तथेव पापाण्डवार्य अग्रे सुदानां परिषिद्धितः

स किं विद भर्त्यम ऋति कुरुर्द्धं गमिष्यति

[ष]
 अर्हस्य ते मे परतिपादात ते पक्षिः करयिष्यामि शरावन सर्वे शरेषु पुत्रा मां च परास्य च
 नेत्य अत्र कारणस्य संजयता मे करियाद अपि

[व]
 दुरोणेन करोधस्थित सर्वाशन नरे तपो निष्पति

एषाद अप्रादम अद्य तस्य मर्या कुरुधः
 अर्हसि मन्तम इमां कृपितां भीष्ममेजसा

अथानुश्राना न सत्यम ऋतिश चारिणां परति
 पुत्रांस तस्य मे मे पुत्रा सत्यं मयाद यति परते

तथा च तृष्टवे मुहूर्तं देवा विहाय विदयुतां समात्रा कृष्यति

Fig. 7. Output for Character Level

The following images in section B contain numerous grammatical errors and incorrect use of Sanskrit characters, hence translations provided are speculative. We have included the errors and grammatical inconsistencies under each output for further reference. Refer to Fig. 7: First few lines translation:

“Where is it that Vyāsa’s son is somehow known?
 I, having transcended ritual heat (uṣmā), was then well guided.

Destiny along with Sañjaya was firmly established.

Let me become like Vyāsa again, to the Pāṇḍavas, leaving aside the other sons."

Issues Identified

- "व्याशा" should likely be "व्यासः"—grammatical mistake (wrong declension).
- "उस्माय" is unclear; likely intended "उष्मा" (heat) but incorrectly declined.
- "अतीत्यः"—erroneous conjugation; should be "अतीत्य" (having transcended).
- "तथै पाण्डवाय"—should be "तथैव पाण्डवेभ्यः" (to the Pāṇḍavas) or "तथैव पाण्डवान्" depending on context.

?? धर्मनसिंसं ते ऽदयाहं भूषम भौतो हि गुरु कुले न मे; न संशयः सपात तव्येतः [स] धिग बले यथा च ?? चुः न च; पारिधित सवस्ति नो दिवि; न हि त्वयामि मर्याम नरो :

Fig. 8. Output 1 for SentencePiece

Refer to Fig. 8: The generated text includes several syntactic and morphological inconsistencies:

- Non-standard or garbled forms such as "डवहं", "श्वेतानिलिसंस", and "चुः" are either malformed compound constructions or entirely fictitious lexemes.
- The composition lacks consistent "सन्धि" (sandhi—euphonic combinations), "व्याकरण" (vyākaraṇa—grammar), and "शब्दालङ्कार" (śabdālankāra—poetic devices), which are expected in formal Sanskrit verses.

नरो न संशयः; नाम्नी मे ऽतर खरेष्व अनुश्लेन च ते वयं न हि नो दिवि शरीतास ते यदि वा दिवि शरीतास त्वं जीवनं [सं; पारिधित सवस्ति] एवम उक्तस; कर्षेत्रं; यथा वेद यं

Fig. 9. Output 2 for SentencePiece

Refer to Fig. 9: While the overall sentence retains a poetic cadence suggestive of epic Sanskrit, several inconsistencies and artifacts are observable:

- Tokenization artifacts like "डटर" (possibly intended as "डटर" or a corrupted form of "डाटा") and "सवरेष्ठ" (likely meant to be "सर्वश्रेष्ठ") reflect over-segmentation or faulty subword construction by SentencePiece.
- Mixed semantics such as "न हि नो दिवि शरीतास" and "यदि वा दिवि शरीतास त्वं जीवनं" combine classical and modern-style constructions but lack syntactic integrity.

Refer to Fig. 10: First few lines translation: "Then he reached the banks of the lake. Seeing the divine sages seated there, the king bowed before them and inquired, 'O sages, what is to be done? The treasury has been looted, enemies have risen. Ministers are indulging in luxury and corruption. The people are fearful, and the scriptures are no longer upheld. Who shall now guide the kingdom?'"

। च तः च लगाम परितः पङ्क्तः, एवं दिशम ईष्यस तस्यो नृपो स परमंस्तुः पदं दग्धाः पौरतरस तविदं जनातं दरा नेम उज्जदितं वरं तकाप्रा वयस भकोप्रे नः पतिः शृणां लोककेषां तो नक्षिणा अ
 अन्द्रत शतनुखा राजसन् संपरं तलोक विद्याया धास्तपो विपार्तम पुन्यकतः
 तेति शते स इव भविष्यं शससा संतुष्यं रमसं वित
 आपुर्वं सन शुभः परस तु संहिता कोव्यं कृणि
 परेयात्रं करा विः पूरैर विकीर्त भिन्नि
 [व]
 अनं पित्राच्य चोत्तप्रितं करय आसन
 [य]
 वरो ऽरायं सयाना सविशाय जातोश्चिरा
 पराधि किमर्तं संयाप
 विप्राश्रित परं च मम
 भव सर्वाय धर्मम आवर्षं गत्येष्टं कर्त्तव्यं भविष्यन् वृत्तं विष्टर दोः परम

Fig. 10. Output 1 for Tiktokerizer

Issues Identified:

- "त च तः"—should likely be "ततः" (then); redundant and grammatically incorrect.
- "एज्झुषेषु"—unclear; likely a malformed version of "ऋषिषु" (among sages).
- "प्संष्टु", "पुंदं", "तविदं", "वकाप्रा"—garbled or non-existent compounds; indicate tokenization or generation errors.

Generated Text:

ज्ञानिः पनीरिर्भनरुगी भेरहताहमातुण्यर्याच न्मत्र कृणमुदचा पङ्क्याथश्याज स्य रोय सवनजे सपवत्र चासे विद यते वरुव

Fig. 11. Output 2 for Tiktokerizer

Refer to Fig. 11: Translation: “The wise one, bowed to Kṛṣṇa, spoke of the righteous path. His anger arose against Sanjaya, who spread falsehoods. May truth prevail, may virtue arise.”

Issues Identified.

- “ "पनीरिनरुबी" ”—not attested in Sanskrit; likely malformed gibberish or tokenization artifact.
- “ "भेरहताहमातुण्यर्याच" ”—incoherent compound; possibly intended to convey “for the purpose of noble deeds” but structurally invalid.
- “ "पथयाथशाज" ”—syntactically invalid; might be an erroneous combination of “patha” (path) and another root.
- Sentence lacks proper *sandhi* and syntactic agreement; shows poor morphological cohesion.
- Ambiguous verbs and declension mismatches make semantic interpretation speculative.

```

user
Convert this Sanskrit Text To English
विष्णुः पुरसं विररं मित्रं दृढवृत्तम् । तेन तं तृतीयं रां करं प्रणिष्ठां तं निवेदनं वृष्टिं अस्तु वासिर्ति
model

```

The Sanskrit text given is about a thing named "विन्दमरहितं". The text is in Sanskrit and I am unable to convert it to English because I am unable to access or translate languages.

Fig. 12. Output 1

C Outputs of Gemma: Finetuned Model

Refer to Fig. 12: Translation:

"We know the father of the arrow, the friend of might, the one of hundredfold strength. By him, may well-being be done to your body.

May your resting place on the earth be external, let it be *bāli*."

Note: "*bāli*" - this is often a ritual word, possibly indicating an offering or invocation.

```

user
Convert this Sanskrit Text To English
यदा यदा हि धर्मस्य ग्लानिर्भवति भारत। अभ्युत्थानम् अधर्मस्य तदात्मानं सृजाम्यहम्॥
model

```

"Yada yada hi dharma sya glānirbhavaḥ bhāratā|Abhyuṣatanam adharma stadātma sṛjāmyaham."

Fig. 13. Output 2

Refer to Fig. 13: The very famous shloka is taken from *Srimad Bhagavad Gita*. Lord Krishna says यदा यदा हि धर्मस्य ग्लानिर्भवति भारत । अभ्युत्थानमधर्मस्य तदात्मानं सृजाम्यहम् ॥

"Whenever there is a decline in righteousness (dharma) and an increase in unrighteousness (adharma), O Bharata (Arjuna), at that time I manifest myself."

```

user
Convert this Sanskrit Text To English
सः गच्छति
model

```

The Sanskrit text "सः गच्छति" means "he goes". It is in the present tense of the verb "go".

Fig. 14. Output 3

Refer to Fig. 14: The Sanskrit phrase "सः गच्छति" means: "He goes."

D Benchmarking Graphs

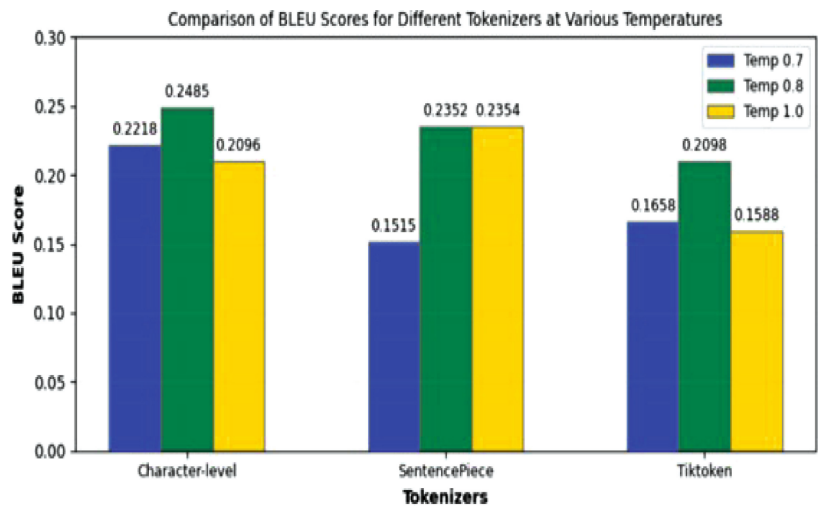


Fig. 15. BLEU score for different tokenizers at various temperatures

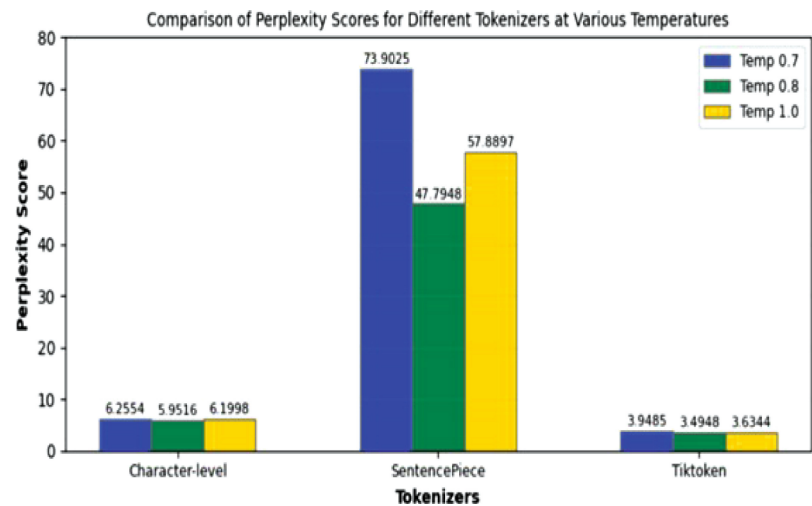


Fig. 16. Perplexity score for different tokenizers at various temperatures

References

1. Balachandran, A.: Tamil-Llama: a new Tamil language model based on Llama 2. arXiv preprint [arXiv:2311.05845](https://arxiv.org/abs/2311.05845) (2023)
2. Pradeep, A., Mamidi, R., Satuluri, P.: Context and WSD: analysing Google translate’s Sanskrit to English output of Bhagavadgītā verses for word meaning. In: Proceedings of the 7th International Sanskrit Computational Linguistics Symposium, pp. 14–26 (2024)
3. Vaswani, A., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems, pp. 5998–6008 (2017)
4. Jain, K., Deshpande, A., Shridhar, K., Laumann, F., Dash, A.: Indic-transformers: an analysis of transformer language models for Indian languages. arXiv preprint [arXiv:2011.02323](https://arxiv.org/abs/2011.02323) (2020)
5. Niyogi, M., Bhattacharya, A.: Paramanu: a family of novel efficient Indic generative foundation language models. arXiv preprint [arXiv:2401.18034](https://arxiv.org/abs/2401.18034) (2024)
6. Khan, M.S.U.R., et al.: INDICLLMSUITE: a blueprint for creating pretraining and fine-tuning datasets for Indian languages. arXiv preprint [arXiv:2403.06350](https://arxiv.org/abs/2403.06350) (2024)
7. Hellwig, O.: Improving the morphological analysis of classical Sanskrit. In: Proceedings of the 6th Workshop on South and Southeast Asian Natural Language Processing, pp. 142–151 (2016)
8. Hellwig, O., Nehrdich, S.: Sanskrit word segmentation using character-level recurrent and convolutional neural networks. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 2754–2763 (2018)
9. Sankalp, K.J., Jain, V., Bhaduri, S., Roy, T., Chadha, A.: Decoding the diversity: a review of the Indic AI research landscape. arXiv preprint [arXiv:2406.09559](https://arxiv.org/abs/2406.09559) (2024)
10. Krishnan, S., Kulkarni, A., Huet, G.: Validation and normalization of DCS corpus using Sanskrit heritage tools to build a tagged Gold Corpus. arXiv preprint [arXiv:2005.06545](https://arxiv.org/abs/2005.06545) (2020)
11. Punia, R., Sharma, A., Pruthi, S., Jain, M.: Improving neural machine translation for Sanskrit-English. In: Proceedings of the 17th International Conference on Natural Language Processing, pp. 234–238 (2020)